

THE COMPLETE REFERENCE · MBB LABS · MAYBANK · SENIOR PRODUCT OWNER

The Ultimate Maybank PO Interview Guide



One document that teaches you to draw design diagrams, write basic SQL, explain the loan lifecycle simply, know every loan type and line of business Maybank serves, master the technical landscape — and build a career plan around the job description.

Six parts, built for this exact role:

- 1 · How to draw design diagrams (step-by-step + a guided practice)
- 2 · SQL basics, hands-on, on a banking schema
- 3 · The loan lifecycle, the easiest way possible
- 4 · Every loan type & line of business Maybank serves
- 5 · All the technical aspects a PO must understand
- 6 · Your career & interview plan (mapped to the JD)

Use it with your other three companions (the beginner story, the JD-aligned pack, and the general lifecycle reference). This is the master index that ties them together.

CONTENTS

What's inside

PART 1 · DESIGN DIAGRAMS

- 1 Why diagrams, and what interviewers look for
- 2 The shapes — your visual alphabet
- 3 How connections work
- 4 The 8-step method
- 5 Zoom levels (the C4 idea)
- 6 Guided practice: build one live
- 7 The finished Maybank diagram + mistakes to avoid

PART 2 · SQL BASICS

- 8 The mental model + a banking schema
- 9 The clauses, joins & aggregates
- 10 Practice questions (with answers)

PART 3 · LOAN LIFECYCLE, MADE EASY

- 11 The whole journey on one page

PART 4 · MAYBANK'S LOANS & LINES OF BUSINESS

- 12 The three lines of business (+ Islamic & Etiqa)
- 13 Every loan / financing product
- 14 Conventional ↔ Islamic, and how the lifecycle bends

PART 5 · THE TECHNICAL ASPECTS

- 15 The technology landscape
- 16 Integration, APIs & events
- 17 Data, MIS & analytics
- 18 Security, cloud & delivery (DevSecOps)
- 19 The PO's technical vocabulary

PART 6 · YOUR CAREER & INTERVIEW PLAN

- 20 Map your resume to the JD
- 21 Gap analysis & your 90-day plan
- 22 Your story, and the final checklist

1 · Why diagrams, and what interviewers look for

You said an interviewer once asked whether you can draw a system design diagram. Here's the reassuring truth: they're not testing whether you can draw *pretty* boxes. They're testing whether you can **take a messy real-world problem and organise it into clear parts and clear connections**. That's a thinking skill, and it's learnable in an afternoon.

IN PLAIN WORDS: WHAT A DESIGN DIAGRAM IS

A system design (architecture) diagram is just a **map of a software system**. It answers two questions: **“What are the pieces?”** (boxes) and **“How do they talk to each other?”** (arrows). That's genuinely all it is. Everything else is detail.

WHAT A GOOD DIAGRAM SHOWS THE INTERVIEWER

- You can **break a big thing into parts** (decomposition).
- You understand **how parts communicate** (and when to use which method).
- You think about the **non-obvious things**: security, scale, what can fail.
- You can **explain it out loud** so a non-expert follows along.

2 · The shapes — your visual alphabet

Like letters make words, a few shapes make any diagram. Use them **consistently**: the same kind of thing always gets the same shape.

Shape vocabulary



User / Actor — a person who uses the system



Service / Application — a running piece of software



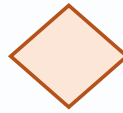
Database — where data is stored



Message queue — a waiting line for events



Cloud — the internet / an outside system



Decision / Router



Gateway (API / payment)



Component / Module

Connections



Synchronous call (asks & waits)



Async event (fire & forget)



Two-way conversation

Memorise these seven shapes and three line styles. They're enough to draw almost any system.

3 · How connections work

The arrows matter as much as the boxes. Every arrow should tell the reader **three** things — train yourself to always say all three out loud:

AN ARROW SHOWS...	BY...	EXAMPLE
Direction — who starts the conversation	which way the arrowhead points	App → Core (the app asks the core)
Type — wait for a reply, or not?	solid line vs dashed line	solid = ask & wait · dashed = send & move on
What & how — the method + purpose	a short text label	“REST/HTTPS – get balance”

IN PLAIN WORDS: SYNCHRONOUS VS ASYNCHRONOUS

Synchronous = the caller **asks and waits** for the answer, like a phone call. Use it when you need the reply *now* (check a balance, approve a payment, return a credit decision the customer is waiting for).

Asynchronous = the caller **sends a message and moves on**, like a text/WhatsApp. Use it for things that can happen in the background (send an SMS, update analytics, tell another system “a loan was booked”). These usually travel through a **queue / event stream** (e.g. Kafka).

SAY THIS — IT SCORES POINTS

“I use **synchronous** calls when the user is waiting on the answer, and **asynchronous events** for background work — that decoupling keeps the core stable and the app fast.” Knowing *when* to use each is what separates a strong candidate from someone who's just memorised words.

Common labels (the “how”): REST/HTTPS gRPC events (Kafka) SQL SFTP file batch ISO 20022 (payments)

4 · The 8-step method

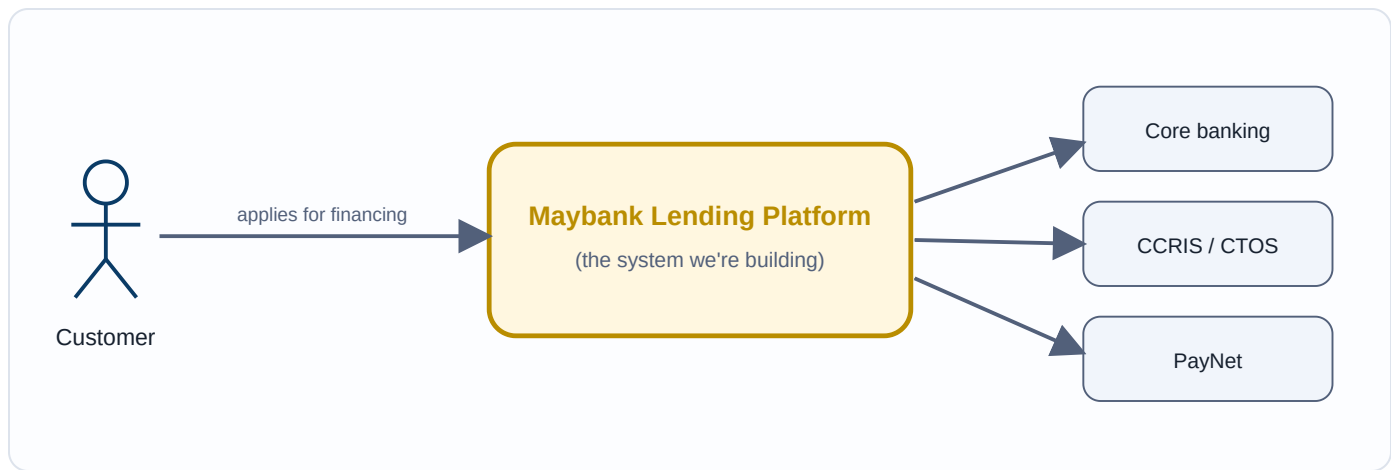
Run these steps in order, out loud, and you'll never freeze at a whiteboard again.

- 1 Clarify the problem first.** What must it do? And the “-ilities”: how many users, how fast, how available, how secure, and what **compliance** (BNM, PCI, PDPA, Shariah)? Saying these before drawing is half the score.
- 2 Put the actors & outside systems at the edges.** Users on the left; external systems (CCRIS, CTOS, PayNet) on the right.
- 3 List the main parts by what they do.** “Origination”, “Credit decisioning”, “Pricing”, “Servicing”. One box = one job.
- 4 Add the data stores.** Each part that remembers things gets a database (cylinder).
- 5 Stack them in layers.** Channels → Security edge → Services → Integration → Core → Data. Layers reveal the flow.
- 6 Draw & label every connection.** Direction + type + purpose. No bare arrows.
- 7 Mark the security boundary.** A dashed box around the “outside-facing” zone; note where login/MFA and encryption sit.
- 8 Walk one journey through it & tidy up.** Trace “a customer applies → ...” and check every hop exists. Add a legend. Done.

5 · Zoom levels (the C4 idea)

Professionals draw at different **zoom levels** depending on the audience — a model called **C4**. Mentioning it makes you sound senior.

LEVEL	ZOOM	SHOWS
1 · Context	Furthest out	The whole system as <i>one</i> box, plus who/what it talks to. Great for non-technical stakeholders.
2 · Container	Closer	The apps, services and databases inside. This is the level you'll usually draw in an interview.
3 · Component	Inside one box	The parts within a single service.
4 · Code	Closest	Classes/functions. Rarely drawn by hand.



A **Context** diagram (Level 1): the simplest possible view. Start here, then zoom in to Container level for the detail.

6 · Guided practice: build one live

Let's draw a diagram together for a realistic prompt, the way you would on the day. **Prompt:** “Design a digital home-financing application for the MAE app.” Follow the 8 steps:

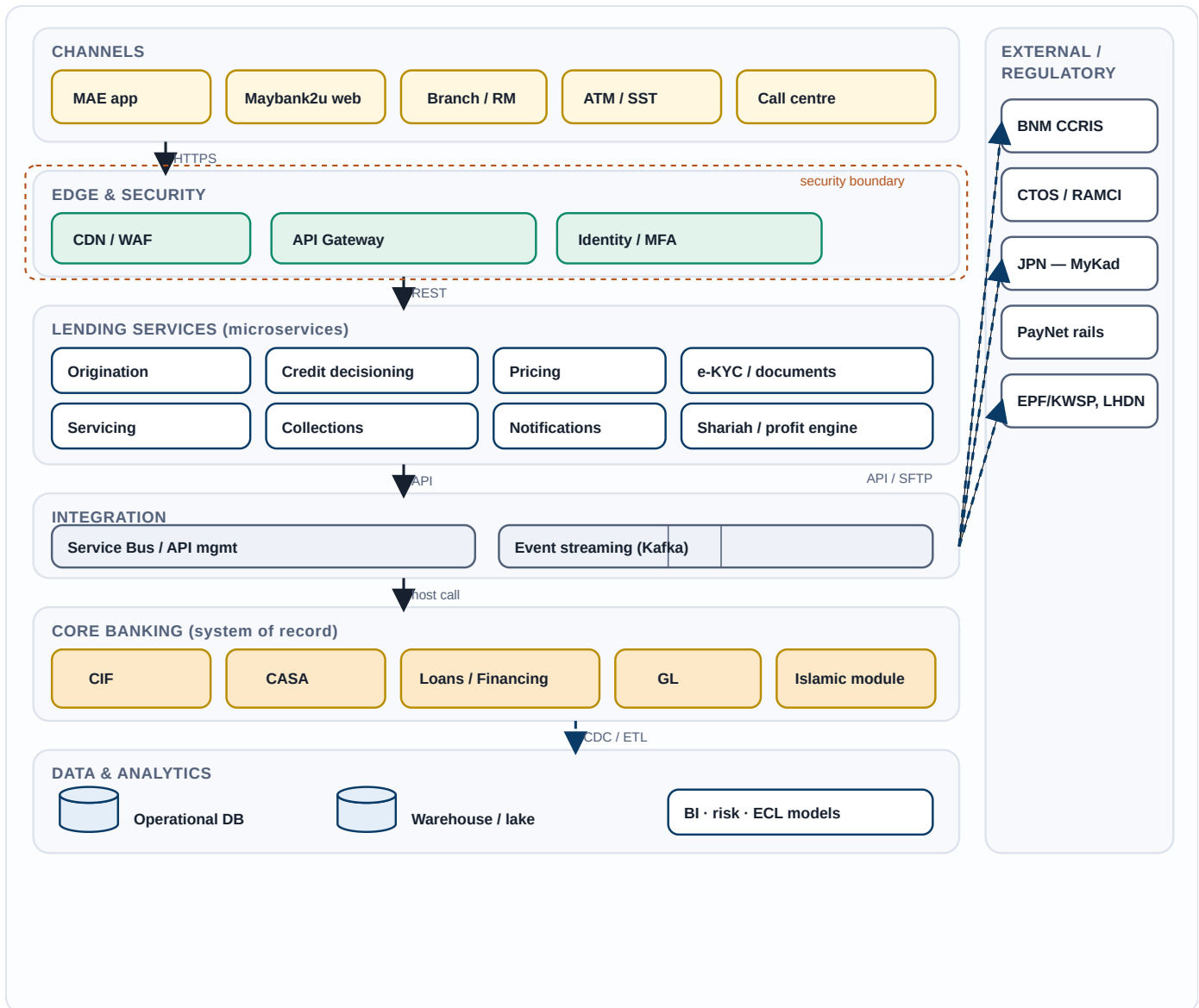
- 1 **Clarify:** “So we want customers to apply for home financing in-app, get a decision in minutes, fully compliant with BNM and PDPA, available 24/7, secure. Peak maybe thousands of applications a day.” (*You just earned marks before drawing anything.*)
- 2 **Edges:** Actor = the customer on MAE. External = CCRIS, CTOS, MyKad (e-KYC), the core banking system, PayNet.
- 3 **Parts by job:** Origination (runs the journey), e-KYC, Credit decisioning, Pricing, Document service, Notifications.
- 4 **Data:** an application database; later, a data warehouse for reporting.
- 5 **Layers:** Channel (MAE) → Edge (gateway + login) → Services (the six above) → Integration (event bus) → Core banking → Data.
- 6 **Connect & label:** MAE → (HTTPS) Gateway → (REST) Origination → (REST) Decisioning → (API) CCRIS/CTOS; Origination → (event) Notifications; Core → (CDC) Warehouse.
- 7 **Security boundary:** dashed box around gateway + login; that's where the outside world meets the inside.
- 8 **Walk it:** “Customer taps Apply → passes the gateway after MFA → Origination calls e-KYC (MyKad) and Decisioning (CCRIS+CTOS, computes DSR) → Pricing returns a rate → on accept, it books to Core → an event tells Notifications to SMS the customer.” Every hop exists. ✓

THE SECRET

Notice you didn't need to memorise an answer — you ran a **method**. The same 8 steps work for *any* prompt they give you (a payments system, a collections tool, a deposit-onboarding flow). Learn the method, not a picture.

7 · The finished Maybank diagram + mistakes to avoid

Here's what that practice produces at **Container** level — a clean, layered, labelled diagram. Study how it reads top-to-bottom.



The finished Container-level diagram. Yellow = the bank's core. Solid arrows = synchronous; dashed = asynchronous/batch.

FIVE MISTAKES THAT LOSE MARKS

1. **Bare arrows** with no labels — always state protocol + purpose.
2. **Jumping into detail** before clarifying requirements.
3. **Forgetting security & compliance** — in a bank this is never optional.
4. **Inconsistent shapes** — pick a style and stick to it.
5. **Drawing in silence** — narrate as you go; the talk is the test.

Tools to mention: draw.io / diagrams.net, Lucidchart, Miro, Excalidraw, Visio, PlantUML / Mermaid

8 · SQL — the mental model + a banking schema

The JD asks for **basic SQL**. As a PO you won't write production queries, but you'll **read** them, ask analysts for data, and sanity-check numbers. That's very learnable.

IN PLAIN WORDS: WHAT SQL IS

SQL (“Structured Query Language”) is how you **ask a database questions**. Data lives in **tables** — like spreadsheets, with **rows** (records) and **columns** (fields). Tables link through **keys**: a **primary key (PK)** uniquely names each row; a **foreign key (FK)** points to another table's PK. SQL is just a polite way of saying “show me the rows where...”.

Our toy schema (a loans database):

TABLE	COLUMNS
customers	customer_id (PK), name, income
loans	loan_id (PK), customer_id (FK), product, outstanding, rate, status, months_in_arrears
payments	payment_id (PK), loan_id (FK), paid_on, amount

9 · The clauses, joins & aggregates

Six clauses do 90% of everything. Read this query like an English sentence:

```
-- "Show me, per product, how many active loans and total owed, biggest first."
SELECT  product, COUNT(*) AS num_loans, SUM(outstanding) AS total_owed
FROM    loans
WHERE   status = 'active'           -- filter rows
GROUP BY product                  -- one result row per product
ORDER BY total_owed DESC          -- sort
LIMIT   10;                       -- keep top 10
```

CLAUSE	DOES
SELECT	which columns to show
FROM	which table
WHERE	filter rows (=, >, IN, BETWEEN, LIKE, IS NULL)
GROUP BY + COUNT/SUM/AVG	roll many rows into summaries
ORDER BY	sort the results
LIMIT	cut to the top N

JOIN — gluing tables together

```
-- Put each customer's name beside their loans
SELECT c.name, l.product, l.outstanding
FROM   customers c
JOIN   loans l ON l.customer_id = c.customer_id;
```

INNER JOIN = only matches in both tables · **LEFT JOIN** = all of the left table + matches (blanks where none — e.g. “all customers, even those with no loan”).

10 · Practice questions (with answers)

Cover the answers and try first. Getting the **shape** right matters more than perfect syntax.

Q1 — EASY

“List all loans that are 3+ months in arrears, biggest balance first.”

```
SELECT loan_id, outstanding, months_in_arrears
FROM   loans
WHERE  months_in_arrears >= 3
ORDER BY outstanding DESC;
```

Q2 — AGGREGATE

“Total amount owed, per product.”

```
SELECT product, SUM(outstanding) AS total
FROM   loans
GROUP BY product;
```

Q3 — THE IMPRESSIVE ONE

“What % of each product's book is impaired (3+ months late)? — the GIL ratio.”

```
SELECT product,
       SUM(CASE WHEN months_in_arrears >= 3 THEN outstanding ELSE 0 END)
       / SUM(outstanding) AS gil_ratio
FROM   loans
GROUP BY product;
```

IF ASKED SQL LIVE, SAY THIS

“I'd GROUP BY product , SUM the outstanding, and join loans to customers on customer_id . For the impaired share I'd use a CASE inside the SUM .” Showing you can **shape** the query is the whole point.

Worth knowing the names of: `DISTINCT` , `HAVING` (filter on an aggregate), `CTE` (`WITH ... AS`), window functions (`ROW_NUMBER()` `OVER(...)`), and DDL/DML (`CREATE/INSERT/UPDATE/DELETE` vs `SELECT`).

11 · The loan lifecycle, the easiest way possible

The simplest way to hold the whole thing in your head: a loan has a **beginning** (getting it), a **middle** (paying it), and an **end** (finishing it). Nine stages, but only three ideas.

PHASE	STAGE	IN ONE LINE
Getting it	1 · Apply	Customer asks; bank collects info & checks ID (KYC).
	2 · Credit check	Can & will they repay? (5 Cs, DSR, CCRIS/CTOS) → a risk rating.
	3 · Approve & price	Yes/no, and the rate (risk-based: SBR + spread).
	4 · Sign & disburse	Sign the promise, register collateral, pay the money out.
Paying it	5 · Servicing	Monthly instalments; each splits into interest + principal.
If it goes wrong	6 · Delinquency	Missed payments; bank chases (collections).
	7 · Rehabilitation	Restructure/reschedule to keep them paying (R&R, AKPK).
	8 · Recovery	Last resort: sell the collateral (foreclosure/repossession).
Finishing it	9 · Settlement	Fully paid; collateral released; closed — a gold star on credit.

THE 10-SECOND VERSION FOR THE INTERVIEW

“Apply → credit-check → approve & price → disburse → repay → and if there's trouble: chase → restructure → recover; otherwise → settle. Most loans only ever touch the beginning, middle and end.” Islamic version: say **financing** not loan, **profit** not interest.

Want the deep, story version? See your companion guide “Lending & Pricing, Explained Like a Story” — it walks one borrower through all nine stages with worked pricing numbers.

12 · Maybank's three lines of business (+ Islamic & Etiqa)

A “**line of business**” (LOB) is just a major division of the bank serving a type of customer. Maybank reports in **three**, with Islamic banking woven across all of them.

LINE OF BUSINESS	SERVES	CORE PRODUCTS
Group Community Financial Services (GCFS)	Everyday people & small businesses (retail, SME, business banking)	Savings/current accounts, fixed deposits, home & personal loans , car financing, cards, SME financing, trade finance, remittance.
Group Global Banking (GGB)	Big corporates, governments, institutions	Corporate lending & syndicated/project finance , investment banking (bond/equity issuance, M&A advisory), global markets/treasury (FX, bonds), transaction banking, asset management.
Group Insurance & Takaful (Etiqa)	Anyone needing protection/cover	Life & general insurance, and family & general takaful (the Islamic version).
CROSS-CUTTING	WHAT IT IS	
Maybank Islamic	The largest Islamic bank in ASEAN . Offers a Shariah-compliant version of nearly every product above — Maybank's “Islamic First” approach. Not a separate LOB so much as a parallel product set across all three.	

SAY THIS

“Maybank runs three lines of business — Community Financial Services for retail & SME, Global Banking for corporates, and Etiqa for insurance & takaful — with **Maybank Islamic** offering a Shariah-compliant version across all of them. A lending product I own usually needs both a conventional and an Islamic variant from day one.”

13 · Every loan / financing product

Here's the catalogue, grouped by who borrows. (Product *names* change over time — verify on maybank2u — but the **types** are stable, and types are what an interview tests.)

Retail — for individuals

PRODUCT TYPE	WHAT IT'S FOR	SECURED?	MAYBANK EXAMPLES
Home / mortgage	Buying property	Yes — the property	MaxiHome, HouzKEY (rent-to-own), home equity
Auto / Hire Purchase	Buying a car (new/used)	Yes — the vehicle	Hire Purchase (up to 90% MOF, up to 9 yrs); Islamic = AITAB
Personal financing	Any personal need (cash)	Usually no	Personal loan, salary/cash financing
Credit cards	Revolving everyday spend	No	Maybank credit/charge cards
Investment financing	Buying unit trusts (very common in Malaysia)	Secured by the units	ASB financing (Amanah Saham), share margin financing
Education financing	Tuition / study	Varies	Education financing

SME & business — for smaller companies

PRODUCT TYPE	WHAT IT'S FOR
SME term financing	Lump-sum for growth/assets (Maybank SME Digital Financing: ~RM3k–RM20m; often unsecured below RM500k).
Working capital / overdraft	Day-to-day cash flow gaps.
Commercial Hire Purchase	Business vehicles & equipment.
Trade finance	Letters of credit, invoice/trade financing for importers/exporters.
Commercial property financing	Shops, factories, offices.

Corporate & institutional — Global Banking

PRODUCT TYPE	WHAT IT'S FOR
Corporate term loans	Large company borrowing.
Syndicated & project finance	Huge loans shared by several banks; financing infrastructure/projects.
Trade & supply-chain finance	Large-scale trade across borders.
Sukuk / bonds	Raising money from investors (sukuk = the Islamic bond).

THE ONE DISTINCTION THAT EXPLAINS THEM ALL

Secured vs unsecured (is there collateral?) and **term vs revolving** (a fixed payoff schedule, or an ongoing credit line?). Almost every product above is just a combination: a mortgage is **secured + term**; a credit card is **unsecured + revolving**; an overdraft is **secured-or-not + revolving**. Master these two axes and the whole catalogue makes sense.

14 · Conventional ↔ Islamic, and how the lifecycle bends

CONVENTIONAL	ISLAMIC EQUIVALENT	CONTRACT USED
Home loan	Home financing-i	Musharakah Mutanaqisah (diminishing partnership) / Commodity Murabahah
Car loan (Hire Purchase)	Hire Purchase-i	AITAB (Al-Ijarah Thumma Al-Bai — lease then buy)
Personal loan	Personal financing-i	Tawarruq (commodity murabahah)
Fixed deposit	Term deposit-i	Tawarruq / Mudarabah
Bond	Sukuk	Various (Ijarah, Murabahah...)

How the lifecycle bends by product type:

- **Secured loans** (home, car) add steps: valuing the collateral, registering the lien, and — if it goes bad — repossession/foreclosure.
- **Unsecured loans** (personal, cards) skip collateral but price higher (no asset to recover), and go straight to charge-off if they fail.
- **Revolving products** (cards, overdraft) don't really “end” — the customer borrows and repays continuously within a limit.

- **Islamic products** follow the same nine stages but every product needs **Shariah Committee** approval and uses profit, not interest.

15 · The technology landscape

You asked to cover “all the technical aspects.” Here's the whole stack a banking PO should be able to talk about — top (what customers touch) to bottom (where money is recorded).

LAYER	WHAT IT IS, IN PLAIN WORDS
Channels	The apps customers use — MAE, Maybank2u, ATMs, branch screens.
Edge & security	The “front door” — a gateway that routes traffic, a firewall (WAF), and login/MFA that checks who you are.
Microservices	The business logic, split into small independent services (origination, pricing, servicing...) instead of one giant program.
Integration	The “plumbing” connecting everything — an API layer plus an event stream (Kafka) for async messages.
Core banking	The master record-book — accounts, loans, the general ledger. The system of record.
Data & analytics	Warehouses/lakes that copy data for reporting, dashboards (MIS) and risk models.

IN PLAIN WORDS: “MICROSERVICES” & “EVENT-DRIVEN”

Microservices = building software as many **small, independent pieces** that each do one job, rather than one huge block. Easier to change one piece without breaking the rest. **Event-driven** = pieces talk by **announcing events** (“a loan was booked!”) that others react to, instead of calling each other directly. This is exactly the “**modern, event-driven architecture**” MBB Labs says it builds — so knowing these two words is gold.

16 · Integration, APIs & events

IN PLAIN WORDS: “API”

An **API** (Application Programming Interface) is a **menu one system offers to another**. Just like you order from a restaurant menu without seeing the kitchen, the MAE app calls the core's API to “get balance” without knowing how the core works inside. APIs are how modern banks let the friendly apps talk safely to the serious core.

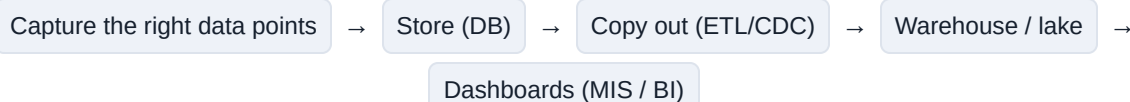
TERM	PLAIN MEANING
REST / HTTPS	The most common way apps call APIs over the web. Synchronous (ask & wait).
Webhook / event	A system pushes a message when something happens (async).
Kafka / event stream	A high-speed “conveyor belt” of events many systems can read.
ESB / API gateway	Middle layer that routes, secures & throttles all this traffic.
Idempotency	A safety trick so a repeated message (e.g. a retried payment) isn't processed twice. <i>Crucial for money.</i>
ISO 20022 / ISO 8583	Standard “languages” for payments and card transactions.

SAY THIS

“Customer-facing apps talk to the core through **APIs**, and services coordinate through **events** so they stay loosely coupled. For anything touching money I'd insist on **idempotency** so a retry never double-charges.”

17 · Data, MIS & analytics

The JD stresses “functional design from data capture to MIS” and “data visualization.” Here's the chain, simply:



TERM	PLAIN MEANING
Data point / data dictionary	An exact field you capture (e.g. income, collateral value) and its definition. If you don't capture it, you can't report on it.
OLTP vs OLAP	The live system that runs the bank (transactions) vs the system that analyses it (reporting). You separate them so reports don't slow the live system.
ETL / CDC	How data is copied from live systems into the warehouse (Extract-Transform-Load; or Change-Data-Capture, which streams changes as they happen).
Data warehouse / lake	Big stores of copied data for analysis. (Warehouse = tidy & structured; lake = raw & flexible.)
MIS / BI	Management Information System / Business Intelligence — the reports & dashboards leaders use to decide. Tools: Power BI, Tableau, Qlik.
The right chart	Trend → line; compare → bar; funnel/drop-off → funnel; status vs target → KPI card.

THE PO INSIGHT WORTH REPEATING

“Design the **report you'll need** first, then work backwards to the **data points to capture**. Reporting fails when nobody captured the field at the start.”

18 · Security, cloud & delivery (DevSecOps)

TOPIC	WHAT A PO MUST KNOW
PCI DSS	The rulebook for handling card data: tokenise card numbers, never store the CVV , encrypt everything, isolate the card environment. Minimise what touches real card data.
Encryption	Scrambling data so only authorised systems can read it — “at rest” (stored) and “in transit” (moving).
IAM / MFA	Identity & Access Management; Multi-Factor Authentication (password + OTP). Controls who can do what.
KYC / AML	Verify identity; screen for money-laundering & sanctions; file suspicious-activity reports. Bake it into the product, don't bolt it on.
PDPA	Malaysia's data-protection law — capture consent, minimise data, protect it.
Cloud	Renting computing from providers (AWS/Azure/GCP) instead of owning servers — scales up/down on demand. Banks adopt it carefully for regulatory reasons.
CI/CD & DevSecOps	Automated building, testing & releasing of software — with security built in from the start. MBB Labs explicitly runs a DevSecOps culture.
NFRs	“Non-functional requirements” — scalability, availability (uptime), latency (speed), resilience. The qualities <i>besides</i> features.

SAY THIS

“For a banking product I treat **security and compliance as design inputs** — PCI for cards, PDPA for data, KYC/AML for onboarding — and I care about the **non-functionals** (uptime, latency, resilience) as much as the features, because trust is the product.”

19 · The PO's technical vocabulary (quick reference)

API

A menu one system offers another to request data/actions.

Core banking

The master record-book of accounts & the ledger.

Microservice

One small, independent piece of software doing one job.

Event-driven

Systems react to announced events rather than calling each other.

Kafka

A high-speed stream/queue carrying events.

Synchronous

Ask and wait (phone call).

Asynchronous

Send and move on (text message).

Idempotency

Safety so a repeat message isn't double-processed.

Latency

How long a request takes — lower is better.

Availability

% of time the system is up (e.g. 99.9%).

Scalability

Coping with more load by adding capacity.

ETL / CDC

Moving data into the warehouse (batch / streaming).

OLTP / OLAP

Running the bank vs analysing it.

Tokenisation

Replacing card numbers with safe stand-ins.

CI/CD

Automated build-test-release pipeline.

NFR

Quality requirements beyond features.

SLA

A promised service level (e.g. 99.9% uptime).

C4

The 4 zoom levels for diagrams.

20 · Map your resume to the JD

ACTION NEEDED — SHARE YOUR RESUME

I built this career section as a **fill-in framework** because I don't have your resume yet — only the job description was shared. **Upload your CV and I'll complete a tailored version** (your exact bullet points mapped to each JD line, a personalised gap plan, and a “why me” pitch in your voice). Meanwhile, work through the templates below — they're useful on their own.

The single most powerful interview prep is a **two-column map**: each JD requirement on the left, **your proof** on the right. Fill this in for yourself:

JD ASKS FOR...	MY EVIDENCE (A PROJECT, RESULT, OR EXAMPLE)
Product ownership & backlog management	
Stakeholder management (international)	
Core banking / lending exposure	
Risk / pricing / analytics	
AGILE + JIRA + Confluence	
Functional design → MIS	
Basic SQL / data	
Leading / mentoring technical teams	

HOW TO FILL EACH BOX

Use a one-line **STAR** result: “Led the backlog for X, cutting onboarding time 30%.” If a box is empty, that's a **gap** to address in §21 — not a reason to panic. Everyone has gaps; what matters is awareness and a plan.

21 · Gap analysis & your 90-day plan

Honest gap analysis

For anything you couldn't fill above, pick a strategy:

IF THE GAP IS...	YOUR MOVE
Knowledge (e.g. banking domain)	Close it now — these four guides are exactly that. Say: “I proactively studied the loan lifecycle and risk-based pricing to prepare.”
A tool (e.g. JIRA, SQL)	Get basic hands-on fast; tools are learnable in days. Show willingness.
Experience (e.g. never built core banking)	Bridge with a transferable example: “I haven't built a core banking system, but I've owned a complex transactional platform where correctness and stakeholders mattered the same way.”

Your first-90-days plan (mention this — it impresses)

PHASE	FOCUS
Days 1–30 · Learn	Understand the product, the core systems, the squad, the stakeholders in KL, and the current backlog. Listen more than talk.
Days 31–60 · Contribute	Take ownership of part of the backlog; ship a small, valuable improvement; build trust with engineering & risk.
Days 61–90 · Lead	Own a roadmap slice end-to-end; set clear metrics; bring a data-backed proposal aligned to ROAR30 priorities.

SAY THIS

“In my first 90 days I'd focus on **learning** the systems and stakeholders, then **contributing** a quick win to build trust, then **owning** a roadmap slice with clear metrics — all anchored to the ROAR30 priorities.”

22 · Your story, and the final checklist

Build your 60-second “why me”

Use this fill-in skeleton (replace the brackets with your truth):

YOUR PITCH TEMPLATE

“I'm a product person with [X years] building [type of products]. I'm strong at [your top 2 strengths — e.g. **backlog ownership & stakeholder alignment**], shown when I [one proud result with a number]. I've deliberately built my foundation in **lending, core banking and risk-based pricing** for this role, and I'm genuinely excited by **ROAR30** and MBB Labs' mandate to modernise with data and AI. I want to own products end-to-end while keeping the customer and ‘Humanising Financial Services’ at the centre.”

Your three STAR stories (prepare these)

1. A **delivery win** — you shipped something valuable (→ Excellence & Efficiency).
2. A **stakeholder conflict** you navigated (→ Relationship Building).
3. A time you **led/mentored or handled ambiguity** (→ Growth, Teamwork).

The everything checklist

✓	I CAN...
☐	Draw a layered system diagram using the 8-step method, narrating as I go.
☐	Explain sync vs async and label every connection.
☐	Read & shape a basic SQL query (SELECT / WHERE / GROUP BY / JOIN).
☐	Tell the loan lifecycle in 10 seconds, and deeply if pushed.
☐	Explain risk-based pricing (5 ingredients; Expected Loss = $PD \times LGD \times EAD$).
☐	Name Maybank's 3 lines of business and the main loan types.
☐	Explain core banking + the digital layer, microservices, events, APIs.
☐	Quote the mission (Humanising), values (T.I.G.E.R.), strategy (ROAR30).
☐	Map my resume to the JD, with a 90-day plan and 3 STAR stories.
☐	Ask 3 thoughtful questions back.

FINAL WORD

You now have the diagrams, the SQL, the lifecycle, the full product map, the technical vocabulary, and a career plan — in one place. The knowledge is handled. On the day, slow down, think out loud, tie answers to the customer and to Maybank's mission, and be honest about what you're still learning. That combination wins. 🐅

Educational interview-prep material. Product names & LOB structure researched from public Maybank sources as of mid-2026 and may change — verify on maybank2u.com. Pricing figures elsewhere are illustrative. Not legal, financial, or Shariah advice.